



Week 3 SOC Internship Report

pfSense, Ubuntu SOC Server, Syslog-ng and Wazuh Integration

Prepared By	Abdullah Masood
Program	Blue Team Internship
Organization	ITSimplera Institute
Reviewed By	Instructor Zia Ullah
Date	11 July 2026

1. Introduction

A Security Operations Center, or SOC, is the part of a company that watches over computers and networks to find security problems early. During week 3 of my internship, I built a small SOC environment on my own computer using VMware Workstation. The goal of this lab was to learn, in a hands-on way, how a firewall, a Linux server, a log collector, and a monitoring tool work together to keep an eye on network activity.

For this exercise I used four virtual machines: a pfSense firewall, an Ubuntu server that acted as the central logging point, a Wazuh server for security monitoring, and a Kali Linux machine to represent another device on the network. I also used a Windows host to access the different web pages of the lab. All of these machines were connected together through virtual networks created inside VMware, so that traffic and logs could move between them just like in a real company network.

The main idea behind the lab was simple: the pfSense firewall creates logs whenever something happens on the network, those logs are sent to the Ubuntu server through a program called Syslog-ng, and then the Wazuh platform reads those logs and shows them on a dashboard so that a security analyst can study them. This report explains, step by step, how I built this setup, the commands I used, the problems I faced, and what I learned from the whole exercise.

2. Objectives

The objectives of this lab were:

- Install and set up a pfSense firewall inside a virtual machine.
- Create and configure networking so that all virtual machines could talk to each other.
- Deploy an Ubuntu server that would act as the central SOC server.
- Install and configure Syslog-ng on the Ubuntu server to receive firewall logs.
- Configure pfSense to send its logs to the Ubuntu server using remote logging.
- Install Wazuh and connect it with the Ubuntu server.
- Register the Ubuntu machine as a Wazuh agent so that it could be monitored.
- Confirm that firewall logs were actually reaching the Ubuntu server and Wazuh.
- Find and fix any errors that came up during installation or configuration.
- Verify, at the end, that every part of the system was working and communicating correctly.

3. Architecture

The lab environment was made up of several virtual machines running together inside VMware Workstation. The pfSense virtual machine acted as the firewall and gateway for the small network. Behind it sat the Ubuntu SOC server, which received and stored logs. The Wazuh server ran the monitoring platform and dashboard. A Kali Linux machine and a Windows host were also part of the network, representing normal devices that could be watched by the SOC.

Traffic and logs moved through the environment in a clear path. First, pfSense generated firewall logs whenever traffic passed through it. These logs were then forwarded, using the Syslog-ng service, from pfSense to the Ubuntu server over the network. On the Ubuntu server, a Wazuh agent watched the log file and reported any new events to the Wazuh Manager. Finally, the Wazuh Manager displayed all of this information on a web dashboard, which I opened from the Windows host using a browser.

Three separate virtual networks were used inside VMware so that the machines could reach each other while still keeping the lab isolated from my main computer network. This matched how a real SOC is normally built, where the firewall, the log server, and the monitoring server are placed on connected but controlled network segments.

Tools and Technologies Used

Before going through the implementation steps, it is useful to briefly describe the main tools used in this lab and the role each one played:

VMware Workstation

VMware Workstation is virtualization software that allows several virtual machines to run on a single physical computer. It was used here to create and run all four machines in the lab, and to build the virtual networks that connected them together, without needing separate physical hardware for each device.

pfSense

pfSense is a free and open-source firewall and router platform. It is normally installed on its own machine and is managed through a web browser. In this lab it acted as the network gateway and firewall, and its ability to forward its own logs to a remote server made it a good source of real security-related log data.

Ubuntu Server

Ubuntu is a popular Linux distribution, and its server edition is commonly used to run background services without a graphical desktop. In this lab, Ubuntu was used as the central point that collected logs from pfSense and hosted the Wazuh agent that reported those logs onward to the Wazuh Manager.

Syslog-ng

Syslog-ng is a log management program that can receive, filter, and store log messages sent over a network using the syslog protocol. It was configured on the Ubuntu server to listen on UDP port 514 and write incoming pfSense log messages into a local file that Wazuh could later read.

Wazuh

Wazuh is an open-source security monitoring platform, often described as a SIEM (Security Information and Event Management) tool. It is made up of a Manager, which collects and analyzes data, and Agents, which are installed on the machines being monitored. Wazuh also provides a web dashboard where an analyst can search, filter, and review security events, which was used here to confirm that pfSense activity was being captured correctly.

4. Implementation Summary

The main steps I followed to complete this lab are listed below, and each one is explained in more detail in the sections that follow:

1. Created the VMware virtual machines needed for the lab.
2. Installed and configured the pfSense firewall.
3. Set up networking between all the virtual machines.
4. Installed and configured the Ubuntu SOC server.

5. Installed Syslog-ng on Ubuntu to receive log messages.
6. Configured Syslog-ng to listen for incoming logs on UDP port 514.
7. Enabled remote logging on pfSense so it would send logs to Ubuntu.
8. Checked the log file on Ubuntu to confirm pfSense logs were arriving.
9. Installed Wazuh and opened its web dashboard.
10. Registered the Ubuntu server as a Wazuh agent.
11. Edited the Wazuh configuration file so it would read the pfSense log file.
12. Restarted all the required services and verified that everything was active.
13. Used the Threat Hunting page in Wazuh to check that security events were being received.

5. VMware Configuration

The first task was to prepare the virtual machines that the whole lab would run on. I opened VMware Workstation and created the machines I needed for this exercise: the pfSense firewall, the Ubuntu SOC server, the Wazuh server, and a Kali Linux machine. Each virtual machine was given its own virtual disk, memory, and network adapters so that it could run independently while still being able to connect to the others.

After the virtual machines were created, I started the Ubuntu installation. During the setup, I chose the option to install the OpenSSH server, so that I would be able to connect to the machine remotely later using an SSH client instead of always working directly inside the VMware window. I also selected password-based authentication instead of using an SSH key, since this was a lab environment and I wanted a simple setup.

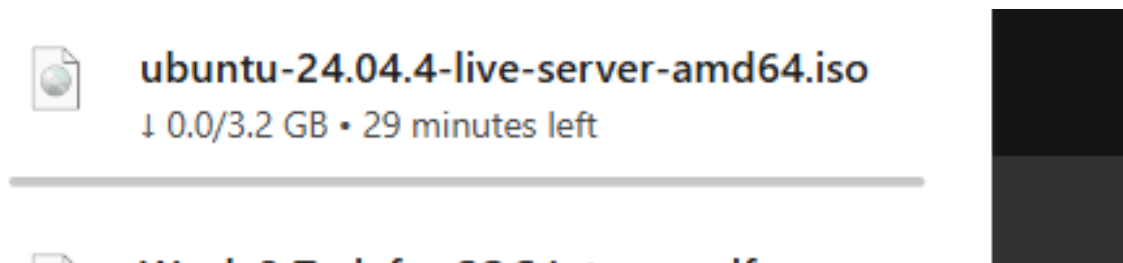


Figure 1: Selecting SSH options during the Ubuntu installation.

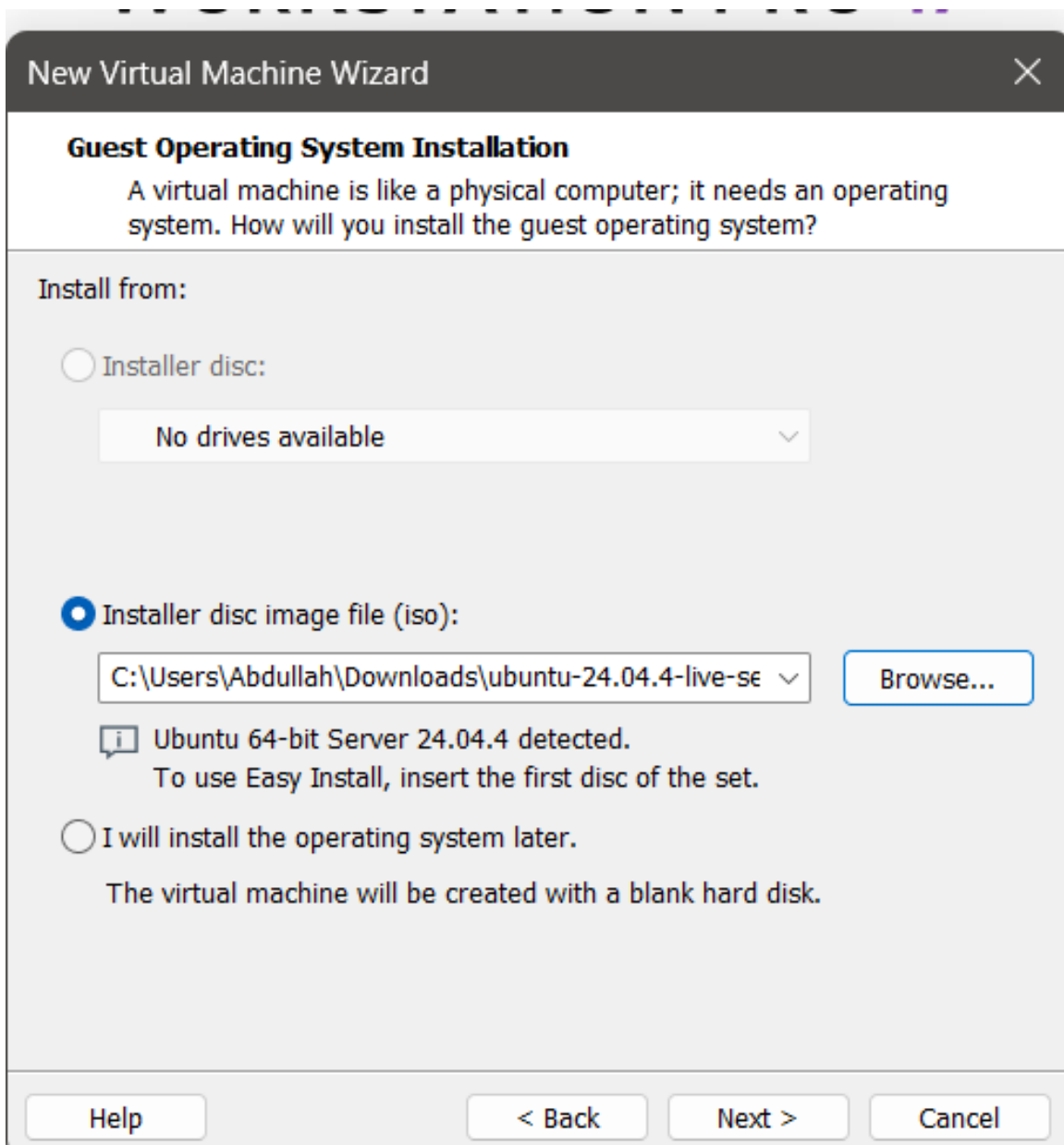


Figure 3: Setting up the user profile during Ubuntu installation.

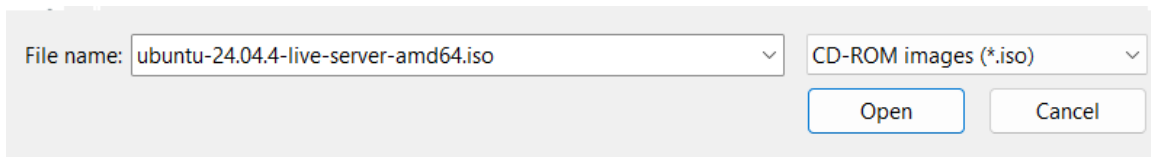


Figure 4: Ubuntu installer moving through the setup steps.

Device	Summary
Memory	4 GB
Processors	2
New CD/DVD (SATA)	Using file C:\Users\Abdullah\...
Network Adapter	NAT
USB Controller	Present
Sound Card	Auto detect
Display	Auto detect

Processors	
Number of processors:	2
Number of cores per processor:	1
Total processor cores:	2

Virtualization engine	
☐	Virtualize Intel VT-x/EPT or AMD-V/RVI
☐	Virtualize CPU performance counters

Figure 5: Disk and storage options shown during installation.

New Virtual Machine Wizard

Name the Virtual Machine

What name would you like to use for this virtual machine?

Virtual machine name:

Location:

The default location can be changed at Edit > Preferences.

Figure 6: Ubuntu installer preparing the system.

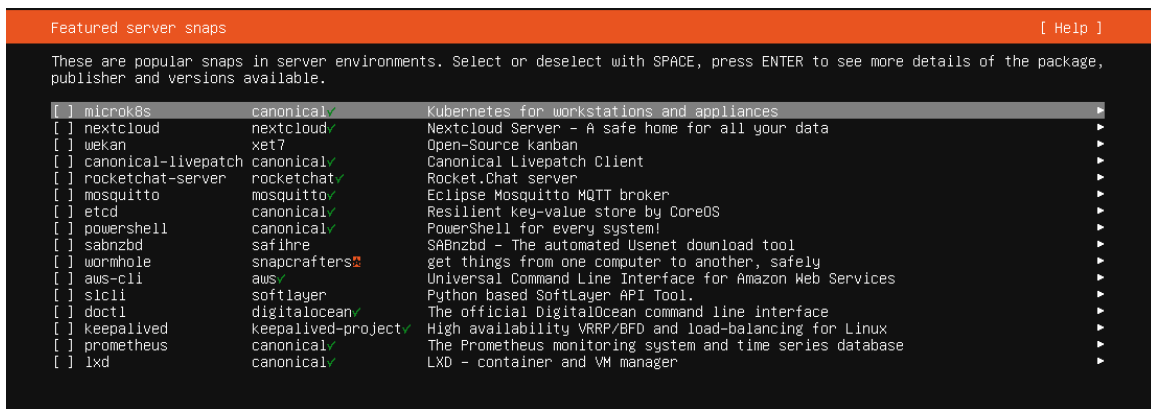


Figure 7: Installation progress screen.

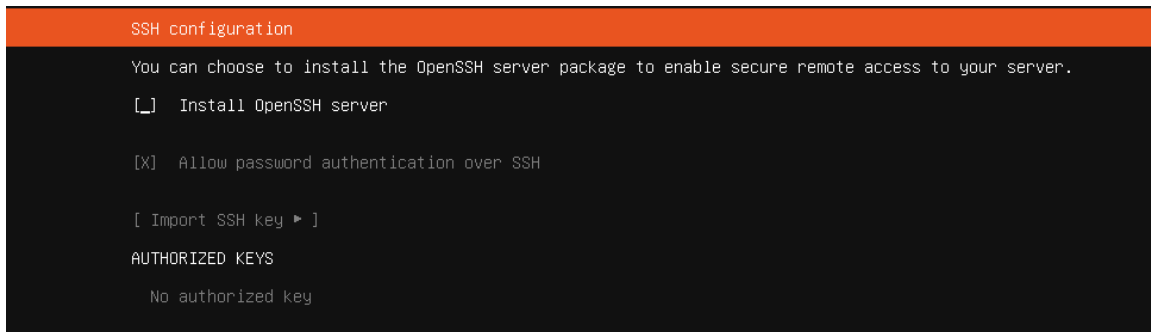


Figure 8: Installation continuing to copy files and set up packages.

Once the base installation steps were finished, the installer showed a final summary confirming which options had been chosen, including the SSH server, password authentication, and the security update settings.

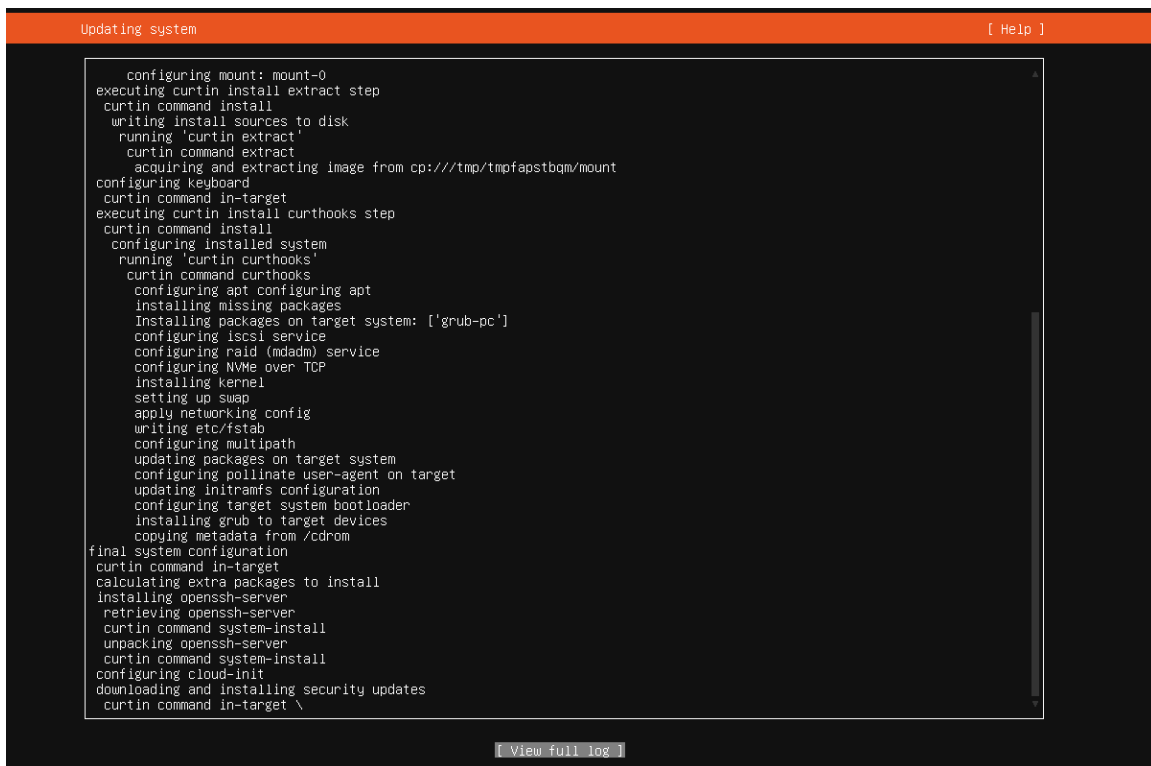


Figure 9: Installation summary screen before the final setup steps.

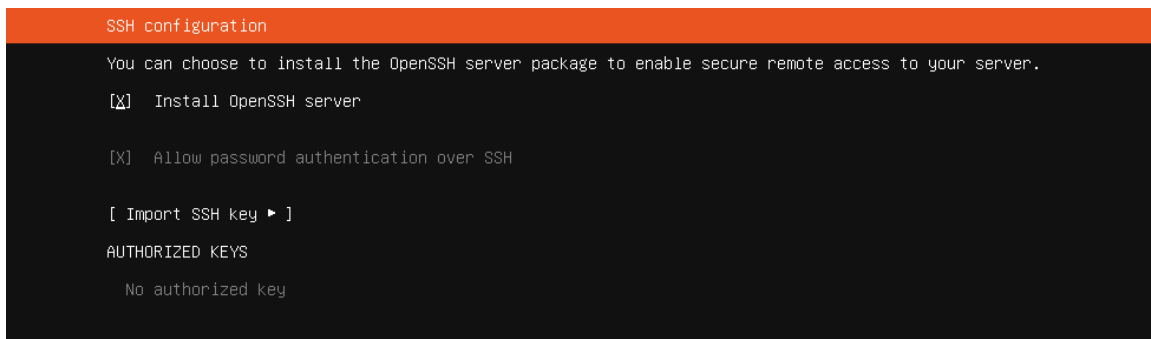


Figure 10: Final SSH configuration confirming OpenSSH server installed with password authentication and no SSH key.

This confirmed that the Ubuntu machine was being prepared correctly and that I would be able to reach it over SSH once it finished booting.

6. Ubuntu SOC Server Setup

After the installation finished, the Ubuntu machine rebooted and reached the login screen. From here, I was able to log in and start preparing the system to act as the central SOC server. Before installing any monitoring software, I needed to install a few supporting packages, including Squid, ClamAV, and Syslog-ng, which would be used later in the lab.

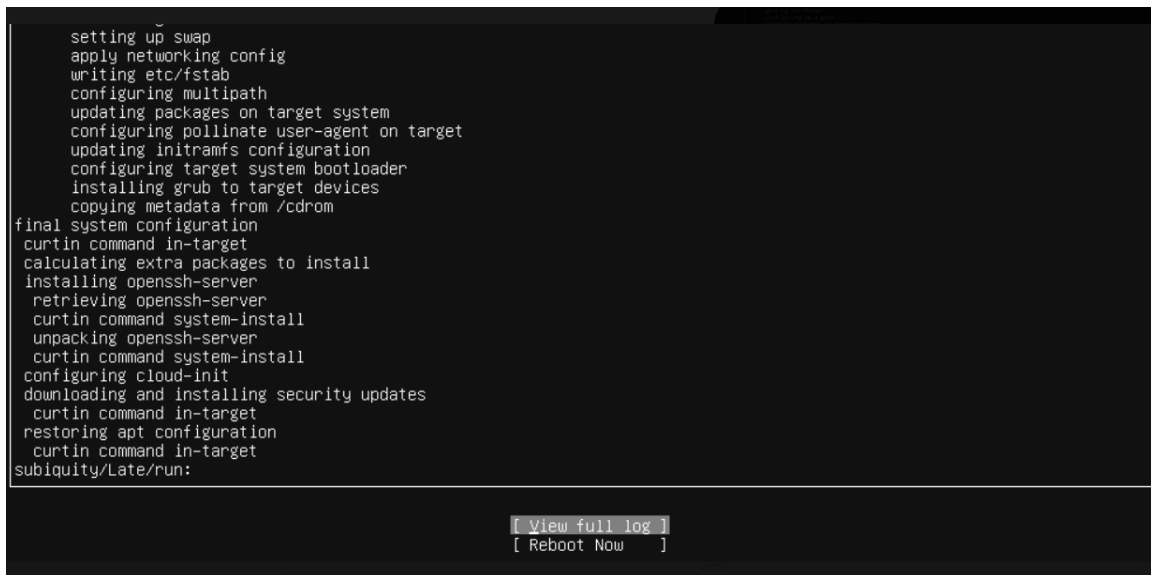


Figure 11: Ubuntu virtual machine installed, with OpenSSH installed and security updates finished.


```

Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-136-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Jul 17 10:03:37 PM UTC 2026

System load:  0.09          Processes:            242
Usage of /:   23.9% of 19.51GB Users logged in:          0
Memory usage: 13%          IPv4 address for ens33: 192.168.204.132
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

36 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

abdullah@ubuntosocserver:~$

```

Figure 12: Ubuntu login screen appearing after the reboot.

Once logged in, I opened a terminal and updated the package lists on the system before installing anything new. This is good practice because it makes sure that the newest and most secure versions of software are installed.

```

abdullah@ubuntosocserver:~$ sudo apt update
[sudo] password for abdullah:
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:3 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,114 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [272 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [286 kB]
Fetched 1,798 kB in 4s (505 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
34 packages can be upgraded. Run 'apt list --upgradable' to see them.
abdullah@ubuntosocserver:~$ _

```

Figure 13: Terminal session on the Ubuntu SOC server after login.

```

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 40.631/41.116/41.496/0.326 ms
abdullah@ubuntosocserver:~$ sudo apt update
[sudo] password for abdullah:
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:3 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,114 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [272 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [286 kB]
Fetched 1,798 kB in 4s (505 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
34 packages can be upgraded. Run 'apt list --upgradable' to see them.
abdullah@ubuntosocserver:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  libdrm-amdgpu1 libfwupd3
The following packages will be upgraded:
  apparmor apport apport-core-dump-handler cloud-init coreutils fwupd iproute2 kpartx libapparmor1 libfwupd2 libjcat1 libnetplan1 libnftables1 libplymouth
  libxmb2 linux-base lshw multipath-tools netplan-generator netplan.io nftables open-vm-tools plymouth plymouth-theme-ubuntu-text python3-apport
  python3-netplan python3-problem-report python3-software-properties rsyslog snapd software-properties-common sosreport systemd-hwe-huob ubuntu-drivers-common
34 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 49.1 MB of archives.
After this operation, 7,110 KB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 coreutils amd64 9.4-3ubuntu6.2 [1,412 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-problem-report all 2.28.2-0ubuntu0.1 [26.0 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-apport all 2.28.2-0ubuntu0.1 [92.9 kB]
Get:4 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 apport-core-dump-handler all 2.28.2-0ubuntu0.1 [18.8 kB]
Get:5 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 apport all 2.28.2-0ubuntu0.1 [85.3 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 ubuntu-drivers-common amd64 1:0.9.7.6ubuntu3.7 [66.5 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 iproute2 amd64 6.1.0-1ubuntu6.4 [1,120 kB]
Get:8 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 open-vm-tools amd64 2:13.0.0-2ubuntu0.24.04.1 [723 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 libapparmor1 amd64 4.0.1really4.0.1-0ubuntu0.24.04.7 [51.3 kB]
Get:10 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 netplan-generator amd64 1.1.2-0ubuntu1~24.04.2 [61.2 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-netplan amd64 1.1.2-0ubuntu1~24.04.2 [24.3 kB]
Get:12 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 netplan.io amd64 1.1.2-0ubuntu1~24.04.2 [69.8 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 libnetplan1 amd64 1.1.2-0ubuntu1~24.04.2 [133 kB]
Get:14 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 rsyslog amd64 8.2312.0-3ubuntu9.2 [511 kB]
14% [14 rsyslog 52.4 kB/511 kB 10%]_

```

Figure 14: Checking basic system details on the Ubuntu server.

I then verified several basic details about the server to make sure everything was set up correctly. The Ubuntu version installed was 24.04.4 LTS, the hostname was set to ubuntosocserver, and the machine had received an IP address of 192.168.204.132 from the NAT network created in VMware. I also tested the internet connection using a ping command, which returned four packets sent and four received with zero packet loss, confirming that the server had working internet access. Finally, I ran the apt update and apt upgrade commands, both of which completed successfully. With these checks passed, the Ubuntu server was ready to be configured as the SOC log collector for the rest of the lab.

7. pfSense Installation

The next machine to configure was the pfSense firewall. pfSense is an open-source firewall and router platform, and in this lab it played the role of the network gateway that all traffic would pass through. I opened the pfSense virtual machine console to check its status and confirm that it had started correctly and picked up its network interfaces.

```

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults 13) Update from console
5) Reboot system             14) Enable Secure Shell (sshd)
6) Halt system               15) Restore recent configuration
7) Ping host                 16) Restart PHP-FPM
8) Shell

Enter an option: 1

Valid interfaces are:

em0    00:0c:29:98:bc:0b    (up) Intel(R) Legacy PRO/1000 MT 82545EM (Copper)
em1    00:0c:29:98:bc:15    (up) Intel(R) Legacy PRO/1000 MT 82545EM (Copper)
em2    00:0c:29:98:bc:1f    (up) Intel(R) Legacy PRO/1000 MT 82545EM (Copper)

Do VLANs need to be set up first?
If VLANs will not be used, or only for optional interfaces, it is typical to
say no here and use the webConfigurator to configure VLANs later, if required.

Should VLANs be set up now [y/n]?

```

```
options=0
groups: pflog
pfsync0: flags=0 metric 0 mtu 1500
options=0
maxupd: 128 defer: off version: 1400
syncok: 1
groups: pfsync
[2.7.2-RELEASE][root@pfsense.localdomain]/root: ifconfig : grep "em0\!em1\!em2\!
inet "
em0: flags=1008843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,LOWER_UP> metric 0 mtu
1500
    inet 192.168.204.131 netmask 0xfffff00 broadcast 192.168.204.255
    inet6 fe80::20c:29ff:fe98:bc0b%em0 prefixlen 64 scopeid 0x1
em1: flags=1008843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,LOWER_UP> metric 0 mtu
1500
    inet 192.168.1.1 netmask 0xfffff00 broadcast 192.168.1.255
    inet6 fe80::20c:29ff:fe98:bc15%em1 prefixlen 64 scopeid 0x2
    inet6 fe80::1:1%em1 prefixlen 64 scopeid 0x2
em2: flags=1008843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,LOWER_UP> metric 0 mtu
1500
    inet 192.168.2.1 netmask 0xfffff00 broadcast 192.168.2.255
    inet6 fe80::20c:29ff:fe98:bc1f%em2 prefixlen 64 scopeid 0x3
    inet 127.0.0.1 netmask 0x0
    inet 10.10.10.1 netmask 0xfffffff
[2.7.2-RELEASE][root@pfsense.localdomain]/root: █
```

Figure 16: pfSense virtual machine console after startup.

From the console, I could see how the pfSense interfaces were mapped to the different networks. The em0 interface was assigned as the WAN connection, linked to the NAT network shared with the rest of the lab. The em1 interface acted as the first LAN, and em2 acted as a second LAN connected to another virtual network used earlier for a related setup.

Once I confirmed the WAN address of pfSense, which was 192.168.204.131, I opened a browser on the Windows host and typed `https://192.168.204.131` into the address bar. After logging in with the pfSense administrator account, the pfSense web-based dashboard appeared, which is the main interface used to manage the firewall.

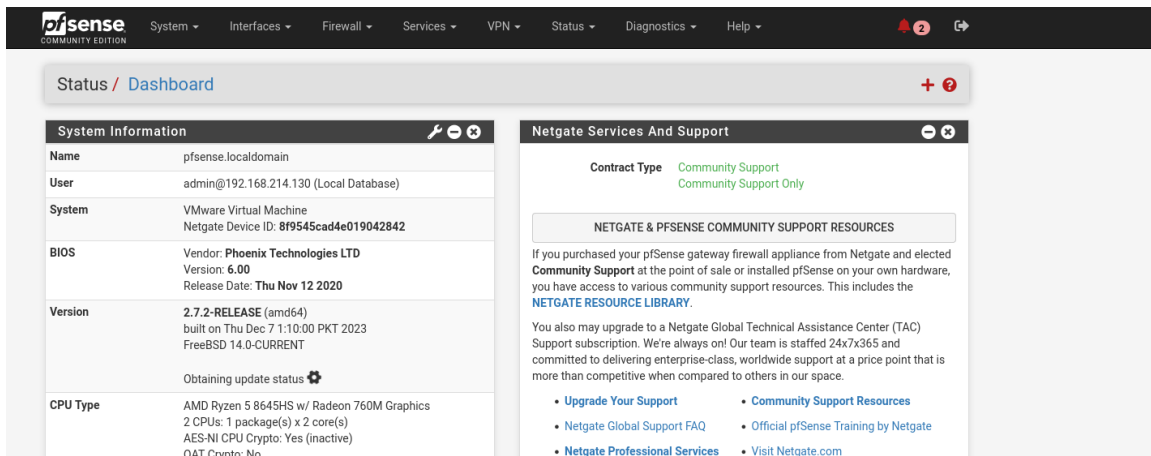


Figure 18: pfSense web interface appearing after a successful login.

8. Network Design

For the different machines to work together, their network adapters had to be set up carefully. I mapped out which VMware network each interface belonged to: the pfSense WAN interface used the NAT network (VMnet8) to reach the internet, the first LAN interface used a host-only network (VMnet1), and the second LAN interface used a custom network (VMnet2) that matched an earlier setup used for another virtual machine.

The most important change at this stage was moving the Ubuntu SOC server's network adapter from the general NAT network to VMnet2, the same custom network used by pfSense's second LAN interface. This step was necessary because the Ubuntu server needed to sit on the same network segment as pfSense so that the two machines could communicate directly and so that firewall logs could later be sent from pfSense to Ubuntu.

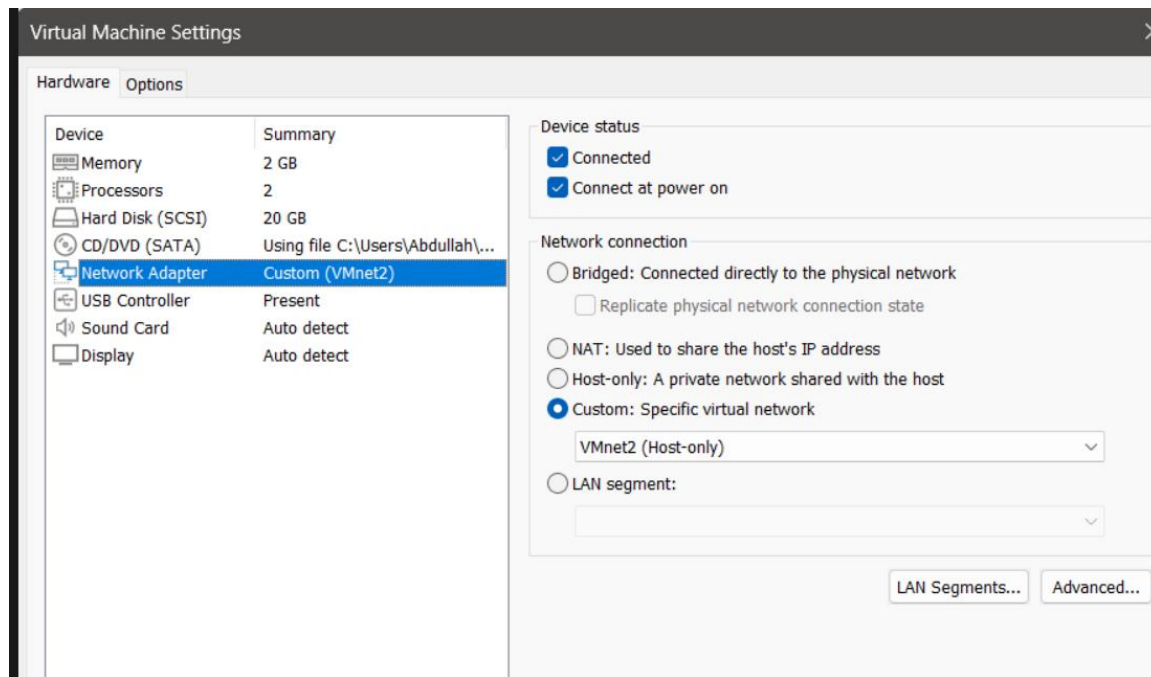


Figure 17:

pfSense interface list showing the WAN and LAN IP addresses assigned to each interface.

```
abdullah@ubuntosocserver:~$ sudo systemctl restart systemd-networkd
[sudo] password for abdullah:
abdullah@ubuntosocserver:~$ networkctl status ens33
• 2: ens33
    Link File: /usr/lib/systemd/network/99-default.link
    Network File: /run/systemd/network/10-netplan-ens33.network
    State: routable (configured)
    Online state: online
    Type: ether
    Path: pci-0000:02:01.0
    Driver: e1000
    Vendor: Intel Corporation
    Model: 82545EM Gigabit Ethernet Controller (Copper) (PRO/10
    Alternative Names: enp2s1
    Hardware Address: 00:0c:29:94:68:af (VMware, Inc.)
    MTU: 1500 (min: 46, max: 16110)
    QDisc: fq_codel
    IPv6 Address Generation Mode: eui64
    Number of Queues (Tx/Rx): 1/1
    Auto negotiation: yes
```

Figure 19: Ubuntu network settings showing the DHCP address, gateway, and DNS server received after switching to the shared network.

After changing the network adapter, I checked the new IP configuration on Ubuntu and confirmed that it had received an address, gateway, and DNS server all pointing to the same subnet used by pfSense. To make sure the two machines could really see each other, I ran a ping test from the Ubuntu server toward the pfSense LAN address.

```
memory usage: 14%          IPv4 address for ens33: 192.168.214.129
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

abdullah@ubuntosocserver:~$ ping -c 4 192.168.214.1
PING 192.168.214.1 (192.168.214.1) 56(84) bytes of data.
64 bytes from 192.168.214.1: icmp_seq=1 ttl=64 time=2.83 ms
64 bytes from 192.168.214.1: icmp_seq=2 ttl=64 time=1.29 ms
64 bytes from 192.168.214.1: icmp_seq=3 ttl=64 time=1.53 ms
64 bytes from 192.168.214.1: icmp_seq=4 ttl=64 time=1.20 ms

--- 192.168.214.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.204/1.712/2.834/0.658 ms
abdullah@ubuntosocserver:~$
```

Figure 20: Successful ping test between the Ubuntu server and the pfSense firewall, confirming network connectivity.

```
abdullah@ubuntosocserver:~$ sudo tcpdump -ni ens33 icmp
[sudo] password for abdullah:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:21:25.638325 IP 192.168.214.1 > 192.168.214.129: ICMP echo request, id 14158, seq 0, length 64
00:21:25.638378 IP 192.168.214.129 > 192.168.214.1: ICMP echo reply, id 14158, seq 0, length 64
00:21:26.641972 IP 192.168.214.1 > 192.168.214.129: ICMP echo request, id 14158, seq 1, length 64
00:21:26.642033 IP 192.168.214.129 > 192.168.214.1: ICMP echo reply, id 14158, seq 1, length 64
00:21:27.644321 IP 192.168.214.1 > 192.168.214.129: ICMP echo request, id 14158, seq 2, length 64
00:21:27.644373 IP 192.168.214.129 > 192.168.214.1: ICMP echo reply, id 14158, seq 2, length 64
00:21:28.677592 IP 192.168.214.1 > 192.168.214.129: ICMP echo request, id 14158, seq 3, length 64
00:21:28.677640 IP 192.168.214.129 > 192.168.214.1: ICMP echo reply, id 14158, seq 3, length 64
00:29:06.437679 IP 192.168.214.129 > 192.168.214.1: ICMP echo request, id 1642, seq 1, length 64
00:29:06.440491 IP 192.168.214.1 > 192.168.214.129: ICMP echo reply, id 1642, seq 1, length 64
00:29:07.439850 IP 192.168.214.129 > 192.168.214.1: ICMP echo request, id 1642, seq 2, length 64
00:29:07.441098 IP 192.168.214.1 > 192.168.214.129: ICMP echo reply, id 1642, seq 2, length 64
00:29:08.441847 IP 192.168.214.129 > 192.168.214.1: ICMP echo request, id 1642, seq 3, length 64
00:29:08.443330 IP 192.168.214.1 > 192.168.214.129: ICMP echo reply, id 1642, seq 3, length 64
00:29:09.444006 IP 192.168.214.129 > 192.168.214.1: ICMP echo request, id 1642, seq 4, length 64
00:29:09.445161 IP 192.168.214.1 > 192.168.214.129: ICMP echo reply, id 1642, seq 4, length 64
```

Figure 21: Network configuration details reviewed on the Ubuntu server after the connectivity test.

With this test passing, I knew that the Ubuntu server and the pfSense firewall could reach each other over the network, which was a required step before any logs could be forwarded between them.

9. Syslog-ng Installation

Syslog-ng is the program that would allow the Ubuntu server to receive log messages sent over the network from pfSense. Before installing it, I also installed Squid and ClamAV, since these were part of the packages required for this stage of the internship task. All three programs were installed together using a single apt command.


```

enable-systemd: uri
abdullah@ubuntuocserver:~$ sudo systemctl status squid
● squid.service - Squid Web Proxy Server
   Loaded: loaded (/usr/lib/systemd/system/squid.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-18 01:17:05 UTC; 4min 34s ago
     Docs: man:squid(8)
    Main PID: 2996 (squid)
      Tasks: 4 (limit: 2205)
    Memory: 16.5M (peak: 17.4M)
       CPU: 547ms
    CGroup: /system.slice/squid.service
            └─2996 /usr/sbin/squid --foreground -sVC
               └─2999 "(squid-1)" --kid squid-1 --foreground -sVC
                  └─3000 "(logfile-daemon)" /var/log/squid/access.log
                     └─3001 "(pinger)"

Jul 18 01:17:05 ubuntuocserver squid[2999]: Using Least Load store dir selection
Jul 18 01:17:05 ubuntuocserver squid[2999]: Set Current Directory to /var/spool/squid
Jul 18 01:17:05 ubuntuocserver squid[2999]: Finished loading MIME types and icons.
Jul 18 01:17:05 ubuntuocserver squid[2999]: HTTP Disabled.
Jul 18 01:17:05 ubuntuocserver squid[2999]: Pinger socket opened on FD 14
Jul 18 01:17:05 ubuntuocserver squid[2999]: Squid plugin modules loaded: 0
Jul 18 01:17:05 ubuntuocserver squid[2999]: Adaptation support is off.
Jul 18 01:17:05 ubuntuocserver squid[2999]: Accepting HTTP Socket connections at conn3 local=[::]:3128 remote=[::] FD 12 flags=9
                                listening port: 3128
Jul 18 01:17:05 ubuntuocserver systemd[1]: Started squid.service - Squid Web Proxy Server.
Jul 18 01:17:06 ubuntuocserver squid[2999]: storeLateRelease: released 0 objects
abdullah@ubuntuocserver:~$ _

```

Figure 26: Final part of the installation output before the command finished.

Once the installation was complete, I checked the status of each service to make sure they had started correctly. The systemctl status command was used to confirm that Squid, Syslog-ng, and ClamAV's daemon were all active and running.

```

https://docs.Clamav.net/
Jul 18 01:17:09 ubuntuocserver systemd[1]: clamav-daemon.service - Clam AntiVirus userspace daemon was skipped because of an unmet condi
abdullah@ubuntuocserver:~$ sudo systemctl status syslog-ng
● syslog-ng.service - System Logger Daemon
   Loaded: loaded (/usr/lib/systemd/system/syslog-ng.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-18 01:17:02 UTC; 8min ago
     Docs: man:syslog-ng(8)
    Main PID: 2798 (syslog-ng)
      Tasks: 2 (limit: 2205)
    Memory: 26.6M (peak: 27.1M)
       CPU: 565ms
    CGroup: /system.slice/syslog-ng.service
            └─2798 /usr/sbin/syslog-ng -F

Jul 18 01:17:02 ubuntuocserver systemd[1]: Starting syslog-ng.service - System Logger Daemon...
Jul 18 01:17:02 ubuntuocserver (syslog-ng)[2798]: syslog-ng.service: Referenced but unset environment variable evaluates to an empty str
Jul 18 01:17:02 ubuntuocserver syslog-ng[2798]: [2026-07-18T01:17:02.164469] smart-multi-line: error opening smart-multi-line.fsm file;
Jul 18 01:17:02 ubuntuocserver syslog-ng[2798]: [2026-07-18T01:17:02.164536] smart-multi-line: your smart-multi-line.fsm seems to be emp
Jul 18 01:17:02 ubuntuocserver syslog-ng[2798]: [2026-07-18T01:17:02.293953] WARNING: Configuration file format is too old, syslog-ng is
Jul 18 01:17:02 ubuntuocserver syslog-ng[2798]: [2026-07-18T01:17:02.669362] WARNING: Your configuration file uses an obsoleted keyword.
Jul 18 01:17:02 ubuntuocserver systemd[1]: Started syslog-ng.service - System Logger Daemon.
lines 1-18/18 (END)

```

Figure 27: Squid service status showing it is active (running).


```

abdullah@ubuntosocserver:~$ sudo systemctl start clamav-daemon
abdullah@ubuntosocserver:~$ sudo systemctl status clamav-daemon
• clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/usr/lib/systemd/system/clamav-daemon.service; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Sat 2026-07-18 01:27:54 UTC; 19s ago
   TriggeredBy: • clamav-daemon.socket
     Docs: man:clamd(8)
           man:clamd.conf(5)
           https://docs.clamav.net/
   Process: 3788 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 3790 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
   Main PID: 3792 (clamd)
     Tasks: 2 (limit: 2205)
    Memory: 957.9M (peak: 958.2M)
       CPU: 12.250s
   CGroup: /system.slice/clamav-daemon.service
           └─3792 /usr/sbin/clamd --foreground=true

Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> ELF support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> Mail files support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> OLE2 support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> PDF support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> SWF support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> HTML support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> XMLDOCS support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> HWP3 support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> OneNote support enabled.
Jul 18 01:28:06 ubuntosocserver clamd[3792]: Sat Jul 18 01:28:06 2026 -> Self checking every 3600 sec

```

Figure 28: Syslog-ng and ClamAV-daemon services both shown as active (running).

At this point, the Ubuntu server had pfSense connectivity, a working network configuration, and the Syslog-ng service ready to receive log messages. The next step was to bring Wazuh into the picture.

10. Wazuh Deployment

Wazuh is the security monitoring platform used in this lab to collect, read, and display log events. After finishing the Ubuntu and pfSense setup, I opened the Wazuh virtual machine and logged in so that I could connect it with the Ubuntu server.

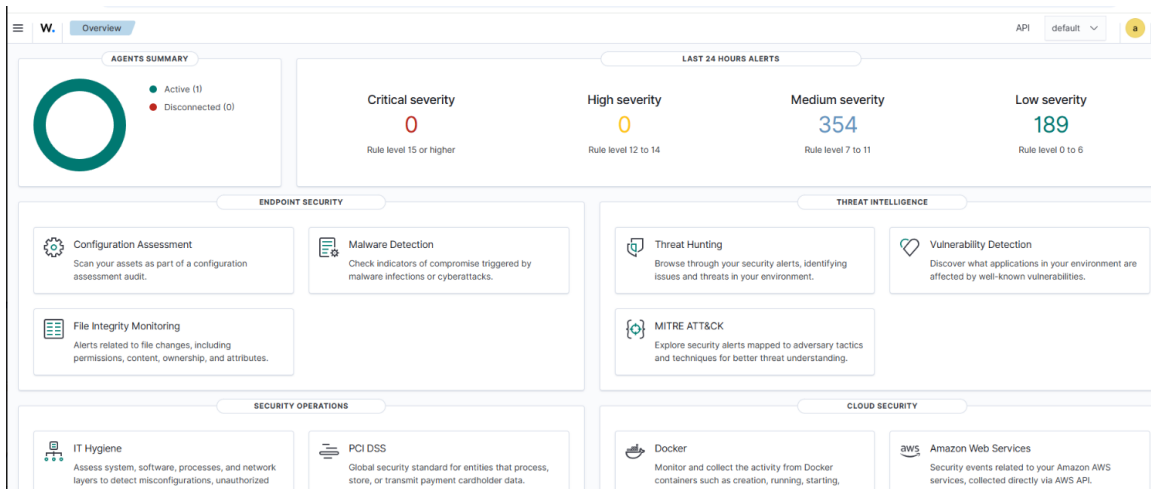


Figure 31: Wazuh login page opened from the Windows browser.

.With the dashboard open, the next task was to make sure that Syslog-ng on Ubuntu and the Wazuh Manager service were both running so that the two systems would be ready to exchange log data.

```

[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-07-18 16:30:41 UTC; 14min ago
     Tasks: 232 (limit: 9442)
    Memory: 2.3G
       CPU: 3min 28.563s
    CGroup: /system.slice/wazuh-manager.service
           └─6249 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              6250 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              6251 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              6254 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              6257 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              6299 /var/ossec/bin/wazuh-authd
              6316 /var/ossec/bin/wazuh-db
              6352 /var/ossec/bin/wazuh-execd
              6372 /var/ossec/bin/wazuh-analysisd
              6438 /var/ossec/bin/wazuh-syscheckd
              6452 /var/ossec/bin/wazuh-remoted
              6462 /var/ossec/bin/wazuh-logcollector
              6506 /var/ossec/bin/wazuh-monitord
              6528 /var/ossec/bin/wazuh-modulesd

Jul 18 16:30:36 wazuh-server env[6185]: Started wazuh-analysisd...
Jul 18 16:30:36 wazuh-server env[6185]: Started wazuh-syscheckd...
Jul 18 16:30:36 wazuh-server env[6185]: Started wazuh-remoted...
Jul 18 16:30:37 wazuh-server env[6185]: Started wazuh-logcollector...
Jul 18 16:30:38 wazuh-server env[6185]: Started wazuh-monitord...
Jul 18 16:30:38 wazuh-server env[6519]: 2026/07/18 16:30:38 wazuh-modulesd:router: INFO: Loaded router module.
Jul 18 16:30:38 wazuh-server env[6519]: 2026/07/18 16:30:38 wazuh-modulesd:content_manager: INFO: Loaded conten
Jul 18 16:30:38 wazuh-server env[6519]: 2026/07/18 16:30:38 wazuh-modulesd:inventory-harvester: INFO: Loaded In
Jul 18 16:30:39 wazuh-server env[6185]: Started wazuh-modulesd...

abdullah@ubuntosocserver:~$ sudo systemctl status syslog-ng
[sudo] password for abdullah:
● syslog-ng.service - System Logger Daemon
   Loaded: loaded (/usr/lib/systemd/system/syslog-ng.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-18 16:40:08 UTC; 2min 20s ago
     Docs: man:syslog-ng(8)
    Main PID: 1241 (syslog-ng)
       Tasks: 2 (limit: 2205)
      Memory: 41.9M (peak: 42.5M)
         CPU: 562ms
        CGroup: /system.slice/syslog-ng.service
                └─1241 /usr/sbin/syslog-ng -F

Jul 18 16:40:07 ubuntosocserver systemd[1]: Starting syslog-ng.service - System Logger Daemon...
Jul 18 16:40:07 ubuntosocserver (syslog-ng)[1241]: syslog-ng.service: Referenced but unset environm
Jul 18 16:40:07 ubuntosocserver syslog-ng[1241]: [2026-07-18T16:40:07.644010] smart-multi-line: er
Jul 18 16:40:07 ubuntosocserver syslog-ng[1241]: [2026-07-18T16:40:07.644298] smart-multi-line: yd
Jul 18 16:40:08 ubuntosocserver syslog-ng[1241]: [2026-07-18T16:40:08.000793] WARNING: Configurati

```

Figure 33: Confirming that Syslog-ng and the Wazuh Manager service are both active on their respective machines.

11. Remote Syslog Configuration

This was one of the more detailed parts of the lab, since it involved changing settings on both pfSense and Ubuntu so that log messages could travel correctly between them. I began by reviewing the pfSense interface IP addresses again to make sure I was using the right address for remote logging.

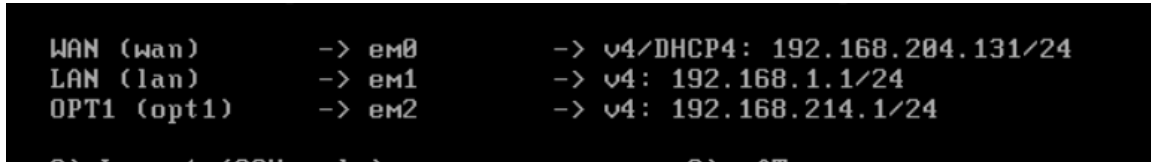


Figure 34: Reviewing the pfSense interface IP addresses before setting up remote logging.

I also had to reconfigure the pfSense firewall virtual machine slightly so that its web interface would keep loading correctly in the browser.

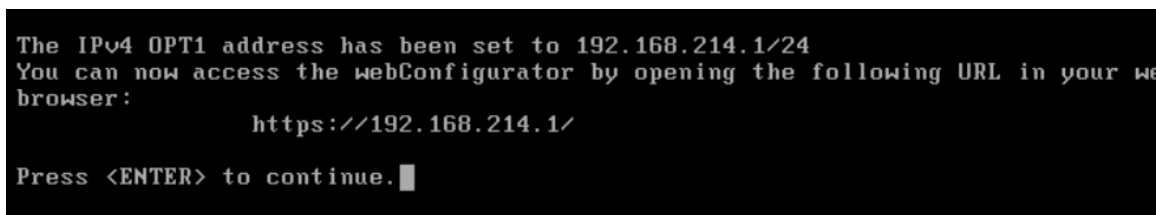


Figure 35: pfSense firewall being reconfigured to keep its web interface reachable.

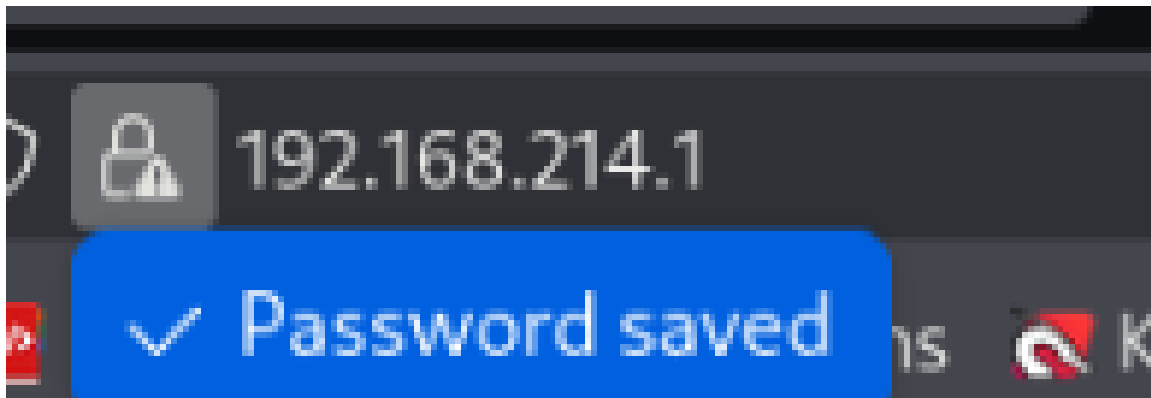


Figure 36: Confirmation that the pfSense web page loaded again after the change.

pfSense
COMMUNITY EDITION

System ▾ Interfaces ▾ Firewall ▾ Services ▾ VPN ▾ Status ▾ Diagnostics ▾ Help ▾

Status / Dashboard

System Information	
Name	pfsense.localdomain
User	admin@192.168.214.130 (Local Database)
System	VMware Virtual Machine Netgate Device ID: 8f9545cad4e019042842
BIOS	Vendor: Phoenix Technologies LTD Version: 6.00 Release Date: Thu Nov 12 2020
Version	2.7.2-RELEASE (amd64) built on Thu Dec 7 1:10:00 PKT 2023 FreeBSD 14.0-CURRENT Obtaining update status ⚙
CPU Type	AMD Ryzen 5 8645HS w/ Radeon 760M Graphics 2 CPUs: 1 package(s) x 2 core(s) AES-NI CPU Crypto: Yes (inactive) QAT Crypto: No

Netgate Services And Support

Contract Type **Community Support**
Community Support

NETGATE & PFSENSE COMMUNITY

If you purchased your pfSense gateway firewall with **Community Support** at the point of sale or installation you have access to various community support resources in the [NETGATE RESOURCE LIBRARY](#).

You also may upgrade to a Netgate Global Technical Support subscription. We're always on! Our team is committed to delivering enterprise-class, world-class support more than competitive when compared to others.

- [Upgrade Your Support](#)
- [Netgate Global Support FAQ](#)
- [Netgate Professional Services](#)
- [Community Support](#)
- [Official](#)
- [Visit](#)

Figure 36: Confirmation that the pfSense web page loaded again after the change.

Log Retention Count

The number of log files to keep before the oldest copy is removed on rotation.

Remote Logging Options

Enable Remote Logging ☐ Send log messages to remote syslog server

Save

Figure 38: pfSense System Logs Settings page, used to enable remote logging.

Inside the pfSense web interface, I went to the Status menu, then System Logs, and finally to the Settings tab. On this page, there is an option to forward system logs to a remote syslog server. I entered the Ubuntu server's IP address here and enabled the remote logging option, so that pfSense would start sending its log messages to port 514 on the Ubuntu server using the UDP protocol.

Selected type is not found on the chosen interface, the other type will be used.

Remote log servers

Remote Syslog Contents

- ☐ Everything
- ☒ System Events
- ☒ Firewall Events
- ☒ DNS Events (Resolver/unbound, Forwarder/dnsmasq, filterdns)
- ☒ DHCP Events (DHCP Daemon, DHCP Relay, DHCP Client)
- ☐ PPP Events (PPPoE WAN Client, L2TP WAN Client, PPTP WAN Client)
- ☒ General Authentication Events
- ☐ Captive Portal Events
- ☐ VPN Events (IPsec, OpenVPN, L2TP, PPPoE Server)
- ☐ Gateway Monitor Events
- ☐ Routing Daemon Events (RADVD, UPnP, RIP, OSPF, BGP)
- ☐ Network Time Protocol Events (NTP Daemon, NTP Client)
- ☐ Wireless Events (hostapd)

Syslog sends UDP datagrams to port 514 on the specified remote syslog server, unless another port is specified. Be sure to set syslogd on the remote server to accept syslog messages from pfSense.

Save

```
GNU nano 7.2
source s_pfsense {
network(
ip("0.0.0.0")
transport("udp")
port(514)
);
};
destination d_pfsense {
file("/var/log/pfsense.log");
};
log {
source(s_pfsense);
destination(d_pfsense);
};
};
```

Figure 39: Editing the Syslog-ng configuration file on the Ubuntu server.

Figure 40: Contents of the Syslog-ng configuration file, set to listen on UDP port 514 and log to the pfSense log file.

After saving the file, I restarted the Syslog-ng service so that the new configuration would take effect, and then checked that Ubuntu was correctly listening for incoming connections on UDP port 514.

```
abdullah@ubuntosocserver:~$ sudo syslog-ng -s
[2026-07-18T18:56:55.988462] smart-multi-line: error opening smart-multi-line.fsm file; filename='/usr/share/syslog-ng/smart-multi-line.fsm'
[2026-07-18T18:56:55.988497] smart-multi-line: your smart-multi-line.fsm seems to be empty or non-existent, automatic multi-line log extension work; filename='/usr/share/syslog-ng/smart-multi-line.fsm'
[2026-07-18T18:56:56.027561] WARNING: Configuration file format is too old, syslog-ng is running in compatibility mode. Please update to 3.38 format at your time of convenience. To upgrade the configuration, please review the warnings about incompatible changes printed by syslog-ng. Change the @version header at the top of the configuration file; config-version='3.38'
[2026-07-18T18:56:56.199446] WARNING: Your configuration file uses an obsolete keyword, please update your configuration; keyword='stats()' block. E.g. stats(freq(1));', location='/etc/syslog-ng/syslog-ng.conf:10:4'
abdullah@ubuntosocserver:~$
```

Figure 40: Contents of the Syslog-ng configuration file, set to listen on UDP port 514 and log to the pfSense log file.

Restarting the Syslog-ng service after editing the configuration.

```
abdullah@ubuntosocserver:~$ sudo systemctl restart syslog-ng
abdullah@ubuntosocserver:~$ sudo ss -ltn | grep 514
UNCONN 0 0 0.0.0.0:514 0.0.0.0:*
abdullah@ubuntosocserver:~$
```

Figure 41: Command output confirming that Ubuntu is listening on UDP port 514.

12. Firewall Logging

With both sides configured, the final step was to save the pfSense remote logging settings and check whether logs were actually arriving on the Ubuntu server. I went back to the pfSense System Logs Settings page and clicked Save to apply the remote logging configuration.

Action	Time	Interface	Rule	Source	Destination	Protocol
✗	Jul 18 17:10:46	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:54841	[2402:4e00::]:53	UDP
✗	Jul 18 17:10:46	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:59417	[2402:4e00::]:53	UDP
✗	Jul 18 17:10:48	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:59417	[2402:4e00::]:53	UDP
✗	Jul 18 17:10:48	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:54841	[2402:4e00::]:53	UDP
✗	Jul 18 17:10:48	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:59417	[2400:3200:baba::1]:53	UDP
✗	Jul 18 17:10:48	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:54841	[2400:3200:baba::1]:53	UDP
✗	Jul 18 17:10:48	LAN	Default deny rule IPv6 (1000000105)	[fe80::8472:694b:f75:aa6a]:59417	[2400:3200::1]:53	UDP

Figure 43: Saving the pfSense remote logging settings, after which firewall logs began to be generated.

After saving the settings, I checked the log file on the Ubuntu server located at `/var/log/pfsense.log`. New entries were appearing in this file, which confirmed that pfSense was successfully sending its firewall logs across the network and that Ubuntu was receiving and storing them correctly. This was an important milestone, since it proved that the log forwarding path between pfSense and Ubuntu was fully working.

13. Agent Registration

The next task was to connect the Ubuntu server to Wazuh as a monitored agent, so that the Wazuh Manager would actually read the pfSense log file and show its contents on the dashboard. This required editing the Wazuh agent configuration file, `ossec.conf`, on the Ubuntu server.

```
sudo nano /var/ossec/etc/ossec.conf
```

Inside this file, I found the last closing tag for the configuration section and added a new block just before it, telling the Wazuh agent to read the pfSense log file as a syslog-formatted source:

```
<localfile>    <log_format>syslog</log_format>
<location>/var/log/pfsense.log</location> </localfile>
```

```
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/active-responses.log</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/pfsense.log</location>
</localfile>

</ossec_config>
```

Figure 44: Editing the ossec.conf file to add the pfSense log file as a monitored source.

```
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-07-18 19:55:01 UTC; 44s ago
     Process: 10562 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 232 (limit: 9442)
   Memory: 1.2G
      CPU: 1min 23.006s
   CGroup: /system.slice/wazuh-manager.service
           └─10626 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─10627 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─10628 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─10631 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─10634 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─10676 /var/ossec/bin/wazuh-authd
                       └─10692 /var/ossec/bin/wazuh-db
                         └─10733 /var/ossec/bin/wazuh-execd
                           └─10749 /var/ossec/bin/wazuh-analysisd
                             └─10810 /var/ossec/bin/wazuh-syscheckd
                               └─10824 /var/ossec/bin/wazuh-remoted
                                 └─10861 /var/ossec/bin/wazuh-logcollector
                                   └─10878 /var/ossec/bin/wazuh-monitord
                                     └─10892 /var/ossec/bin/wazuh-modulesd
```

Figure 45: Confirming that the Wazuh Manager service remained active after the configuration change.

Before registering the agent, I checked the Wazuh VM's own IP address, which was needed to correctly connect the Ubuntu server to the Wazuh Manager.

```
sat 18 19:55:01 wazuh-server eno1:08:00:27:12:34:56:78 completed.
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:aa:d4:3d brd ff:ff:ff:ff:ff:ff
   altname enp2s0
   altname ens32
   inet 192.168.204.130/24 metric 1024 brd 192.168.204.255 scope global dynamic eth0
       valid_lft 1054sec preferred_lft 1054sec
   inet6 fe80::20c:29ff:feaa:d43d/64 scope link link_proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 46: Checking the IP address of the Wazuh virtual machine.

I then installed the Wazuh agent software on the Ubuntu server and pointed it to the Wazuh Manager's IP address so the two systems could communicate.

```

abdullah@ubuntosocserver:~$ sudo WAZUH_MANAGER="192.168.204.130" apt install wazuh-agent -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libestr0 libfastjson4 libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  wazuh-agent
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 13.2 MB of archives.
After this operation, 48.4 MB of additional disk space will be used.
Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.14.6-1 [13.2 MB]
Fetched 13.2 MB in 11s (1,199 kB/s)
Preconfiguring packages ...
Selecting previously unselected package wazuh-agent.
(Reading database ... 92448 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.6-1_amd64.deb ...
Unpacking wazuh-agent (4.14.6-1) ...
Setting up wazuh-agent (4.14.6-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
abdullah@ubuntosocserver:~$

```

Figure 47: Installing the Wazuh agent on the Ubuntu SOC server.

After installation, the Ubuntu server needed to register itself with the Wazuh Manager using a registration key. Once the key was accepted, the Ubuntu server appeared as a properly registered agent.

```

abdullah@ubuntosocserver:~$ sudo /var/ossec/bin/agent-auth -m 192.168.204.130
2026/07/18 20:09:25 agent-auth: INFO: Started (pid: 4370).
2026/07/18 20:09:25 agent-auth: INFO: Requesting a key from server: 192.168.204.130
2026/07/18 20:09:25 agent-auth: INFO: No authentication password provided
2026/07/18 20:09:25 agent-auth: INFO: Using agent name as: ubuntosocserver
2026/07/18 20:09:25 agent-auth: INFO: Waiting for server reply
2026/07/18 20:09:25 agent-auth: INFO: Valid key received
abdullah@ubuntosocserver:~$

```

Figure 48: Valid registration key received, confirming that the Ubuntu SOC server is now registered with the Wazuh Manager.


```
2026/07/18 20:05:23 agentd-auth: INFO: valid key received
abdullah@ubuntosocserver:~$ sudo systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service
abdullah@ubuntosocserver:~$ sudo systemctl restart wazuh-agent
abdullah@ubuntosocserver:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-18 20:11:06 UTC; 31s ago
     Process: 4437 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Process: 4968 ExecReload=/usr/bin/env /var/ossec/bin/wazuh-control reload (code=exited, status=0/SUCCESS)
       Tasks: 33 (limit: 2205)
      Memory: 340.6M (peak: 342.7M)
         CPU: 24.485s
      CGroup: /system.slice/wazuh-agent.service
              └─4467 /var/ossec/bin/wazuh-agentd
                 5304 /var/ossec/bin/wazuh-execd
                 5315 /var/ossec/bin/wazuh-syscheckd
                 5326 /var/ossec/bin/wazuh-logcollector
                 5344 /var/ossec/bin/wazuh-modulesd

Jul 18 20:11:06 ubuntosocserver env[4968]: Killing wazuh-execd...
Jul 18 20:11:07 ubuntosocserver env[4968]: Wazuh v4.14.6 Stopped
Jul 18 20:11:08 ubuntosocserver env[4968]: Starting Wazuh v4.14.6...
Jul 18 20:11:08 ubuntosocserver env[4968]: Started wazuh-execd...
Jul 18 20:11:08 ubuntosocserver env[4968]: wazuh-agentd already running...
Jul 18 20:11:08 ubuntosocserver env[4968]: Started wazuh-syscheckd...
Jul 18 20:11:09 ubuntosocserver env[4968]: Started wazuh-logcollector...
Jul 18 20:11:10 ubuntosocserver env[4968]: Started wazuh-modulesd...
Jul 18 20:11:12 ubuntosocserver env[4968]: Completed.
Jul 18 20:11:13 ubuntosocserver systemd[1]: Reloaded wazuh-agent.service - Wazuh agent.
abdullah@ubuntosocserver:~$
```

Figure 49: Confirmation that the Wazuh agent is installed, authenticated with the Manager, and running as an active service.

Finally, I opened the Wazuh dashboard again to check the list of connected agents. The Ubuntu SOC server appeared in the list as an active agent, alongside other devices in the lab such as the Kali Linux machine and the Windows host.

Agents (3) Deploy new agent Refresh Export formatted More WQL

☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	001	AbdullahPC	192.168.204.1	default	Microsoft Windows 11 Pro 10.0.2200.8875	node01	v4.14.6	● active	ⓘ ⋮
☐	002	Kali	192.168.214.130	default	Kali GNU/Linux 2025.4	node01	v4.14.6	● active	ⓘ ⋮
☐	003	ubuntosocserver	192.168.214.129	default	Ubuntu 24.04.4 LTS	node01	v4.14.6	● active	ⓘ ⋮

Rows per page: 10 < 1 >

Figure 50: Wazuh dashboard confirming that the Ubuntu SOC server agent is shown as active.

14. Threat Hunting

Once the agent was active, I used the Threat Hunting section of the Wazuh dashboard to check whether real security events were flowing in from the Ubuntu server. The dashboard showed three active agents overall, including the Ubuntu SOC server, which confirmed that the full monitoring chain was working.

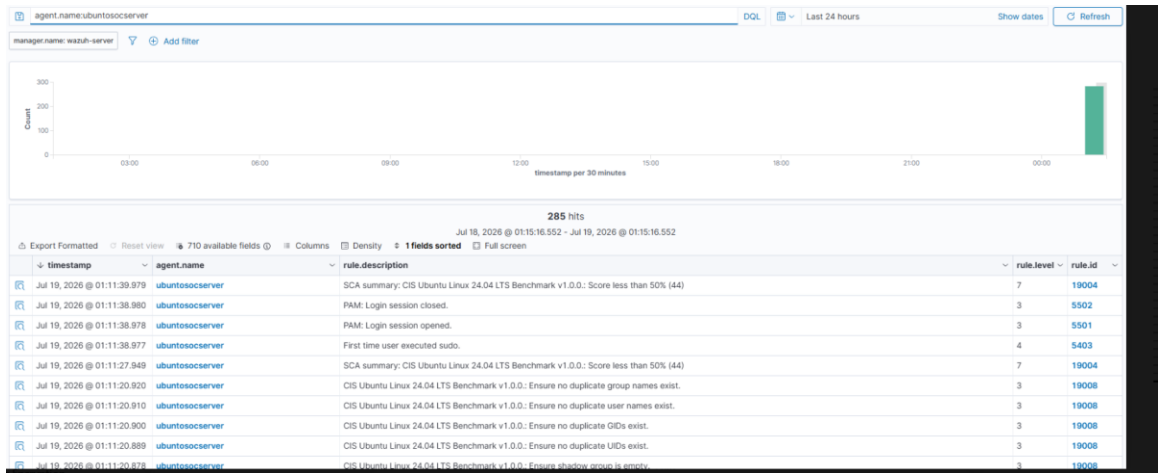


Figure 51: Wazuh dashboard showing three active agents, including the ubuntu socserver agent.

Going into the Security Events page under Threat Hunting, I could see live events being reported by the Ubuntu SOC server agent. This showed that events picked up from the pfSense log file were successfully being turned into readable security alerts inside Wazuh, which is the whole purpose of building this kind of SOC pipeline.

The image shows the Wazuh Ping tool interface. It has a 'Ping' section with configuration options: Hostname (8.8.8.8), IP Protocol (IPv4), Source address (Automatically selected (default)), Maximum number of pings (3), and Seconds between pings (1). Below this is a 'Ping' button. The 'Results' section shows the output of the ping test: PING 8.8.8.8 (8.8.8.8): 56 data bytes, 64 bytes from 8.8.8.8: icmp_seq=0 ttl=128 time=53.264 ms, 64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=48.939 ms, 64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=49.539 ms, --- 8.8.8.8 ping statistics ---, 3 packets transmitted, 3 packets received, 0.0% packet loss, round-trip min/avg/max/stddev = 48.939/50.581/53.264/1.913 ms.

Figure 52: Security Events page showing events reported by the Ubuntu SOC server agent.

15. Testing

To properly test the whole setup, I went back to the pfSense web interface and generated some extra traffic by running another ping test. The idea behind this final test was to make sure that new firewall activity would still be logged and forwarded correctly, and not just the earlier logs I had already seen.

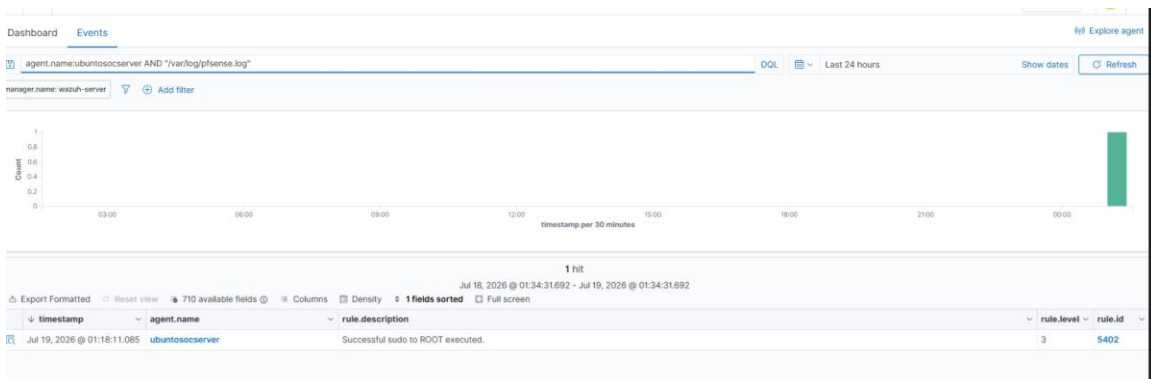


Figure 53: Generating new traffic from pfSense and confirming that new firewall logs are being created.

After this test, I reviewed the complete chain of checks that confirmed the lab was working end-to-end:

- pfSense is generating firewall logs as expected.
- pfSense is successfully sending those logs to the Ubuntu server.
- The file `/var/log/pfsense.log` on Ubuntu is receiving new log entries.
- The Wazuh agent is installed on Ubuntu and is active.
- The Wazuh agent is correctly monitoring the `pfsense.log` file.
- The Ubuntu agent is connected to the Wazuh Manager and reporting events.

16. Troubleshooting

Throughout the lab, a few issues came up that needed to be fixed before moving to the next step. The first challenge was getting the networking right: the Ubuntu server initially sat on a different virtual network than pfSense, which meant the two machines could not talk to each other. This was fixed by changing the Ubuntu network adapter to the same VMware network used by the pfSense LAN interface, after which the ping test between the two machines succeeded.

A second issue appeared while editing configuration files such as the Syslog-ng configuration and the Wazuh `ossec.conf` file. Because these files use a strict format, small typing mistakes could easily stop the related service from starting. Each time a configuration file was changed, I restarted the affected service and checked its status with `systemctl` to make sure it had not failed, and I reviewed the file again whenever an error appeared.

Another point that needed close attention was the pfSense web interface, which stopped loading correctly at one point after a network change. This was solved by re-checking the pfSense virtual machine settings and reconfiguring the affected interface until the web page could be reached again from the browser.

Finally, when registering the Ubuntu server as a Wazuh agent, it was important to use the correct Wazuh Manager IP address and a valid registration key. Any mismatch here would stop the agent from appearing as active on the dashboard, so I double-checked the IP address of the Wazuh VM before running the registration command.

In general, most of the problems in this lab were solved by following the same simple routine: make one change at a time, restart the affected service, check its status, and only move on once that one piece was confirmed to be working. This step-by-step habit made it much easier to find the exact cause of a problem instead of guessing between several possible reasons at once, and it is a habit I plan to keep using in future labs and in real SOC work.

17. Results

By the end of this lab, all the planned objectives had been achieved. The pfSense firewall was installed, configured, and reachable through its web interface. The Ubuntu server was set up correctly, verified for its version, hostname, IP address, and internet access, and had all the required packages installed and running.

Networking between pfSense and Ubuntu was working, confirmed through successful ping tests. Syslog-ng was correctly installed and configured to receive firewall logs from pfSense on UDP port 514, and the `/var/log/pfsense.log` file was actively receiving new log entries.

Wazuh was successfully deployed, its dashboard was reachable from the Windows host, and the Ubuntu server was registered as an active Wazuh agent. The Wazuh agent was configured to read the pfSense log file, and real security events generated by pfSense activity could be seen inside the Wazuh Threat Hunting section, showing that the complete monitoring pipeline was functioning as designed.

Overall, the lab met every objective set out at the start. Each component was tested on its own before being connected to the next one, which meant that by the final stage I could be confident that any event appearing on the Wazuh dashboard had genuinely travelled all the way from a real firewall event on pfSense, through Syslog-ng on Ubuntu, and into the Wazuh Manager, rather than being a leftover test entry from an earlier step.

Summary of Verification Checks

The table below lists the main checks carried out during the lab and their outcome.

Component	Check Performed	Outcome
pfSense	Web interface reachable after login	Successful
Ubuntu Server	Version, hostname, IP, and internet access verified	Successful
Networking	Ping test between Ubuntu and pfSense	Successful
Syslog-ng	Service active and listening on UDP 514	Successful
Remote Logging	pfSense logs appearing in /var/log/pfsense.log	Successful
Wazuh Manager	Dashboard reachable and service active	Successful
Wazuh Agent	Registered and shown as active on Ubuntu	Successful
Threat Hunting	Security events visible from Ubuntu agent	Successful

18. Learning Outcomes

This lab gave me several practical skills and a better understanding of how a SOC actually works behind the scenes:

- I learned how to install and configure a pfSense firewall, including managing its interfaces and enabling remote logging.
- I practiced basic Linux server administration on Ubuntu, including installing packages, checking service status, and editing configuration files with nano.
- I understood how VMware virtual networks can be used to connect several virtual machines together in a controlled lab environment.
- I learned how Syslog-ng works as a log collector and how to configure it to listen for logs over UDP.
- I gained hands-on experience deploying Wazuh, registering an agent, and using its dashboard to view security events.
- I improved my troubleshooting skills by identifying and fixing networking and configuration errors as they appeared.
- I saw, in a practical way, how firewall logs, a log collector, and a SIEM platform like Wazuh fit together to give a SOC analyst visibility into network activity.

19. Conclusion

This week's internship task gave me a complete, hands-on look at how a small Security Operations Center can be built using open-source tools. Starting from an empty set of virtual machines, I was able to install pfSense, configure Ubuntu as a SOC server, connect the two through Syslog-ng, and finally bring in Wazuh to monitor and display security events from the collected logs.

The exercise also taught me that building a working monitoring pipeline is not just about installing software, but about carefully connecting each part together, testing at every stage, and fixing small errors as they come up. By the end of the lab, I had a fully working setup where firewall activity on pfSense could be tracked, forwarded, and analyzed through the Wazuh dashboard, which reflects the basic working principle behind real-world SOC environments.

Going forward, this lab has given me a solid base to build on. In future weeks, I would like to explore adding more log sources to the same Ubuntu server, creating custom alert rules inside Wazuh, and testing how the SOC setup reacts to simulated suspicious activity from the Kali Linux machine, so that I can practice not just building a monitoring pipeline but also actually using it to detect and respond to potential security events.

Appendix: Key Commands Used

For quick reference, the main terminal commands used during this lab are listed below.

sudo apt update && sudo apt upgrade -y

Update package lists and upgrade installed packages on Ubuntu.

sudo apt install squid clamav clamav-daemon syslog-ng -y

Install Squid, ClamAV, and Syslog-ng on the Ubuntu server.

systemctl status squid

Check whether the Squid service is active.

systemctl status syslog-ng

Check whether the Syslog-ng service is active.

systemctl status clamav-daemon

Check whether the ClamAV daemon service is active.

sudo nano /etc/syslog-ng/conf.d/wazuh.conf

Create/edit the Syslog-ng configuration to listen on UDP 514.

sudo systemctl restart syslog-ng

Restart Syslog-ng after changing its configuration.

ss -u -l | grep 514

Confirm Ubuntu is listening for UDP traffic on port 514.

tail -f /var/log/pfsense.log

Watch the pfSense log file on Ubuntu for new incoming entries.

sudo nano /var/ossec/etc/ossec.conf

Edit the Wazuh agent configuration to add the pfSense log file.

systemctl status wazuh-manager

Check whether the Wazuh Manager service is active.

systemctl status wazuh-agent

Check whether the Wazuh agent service is active on Ubuntu.

ip addr

Check the IP address assigned to a virtual machine.

ping <ip-address>

Test network connectivity between two virtual machines.